

Customer 360 Data Integration

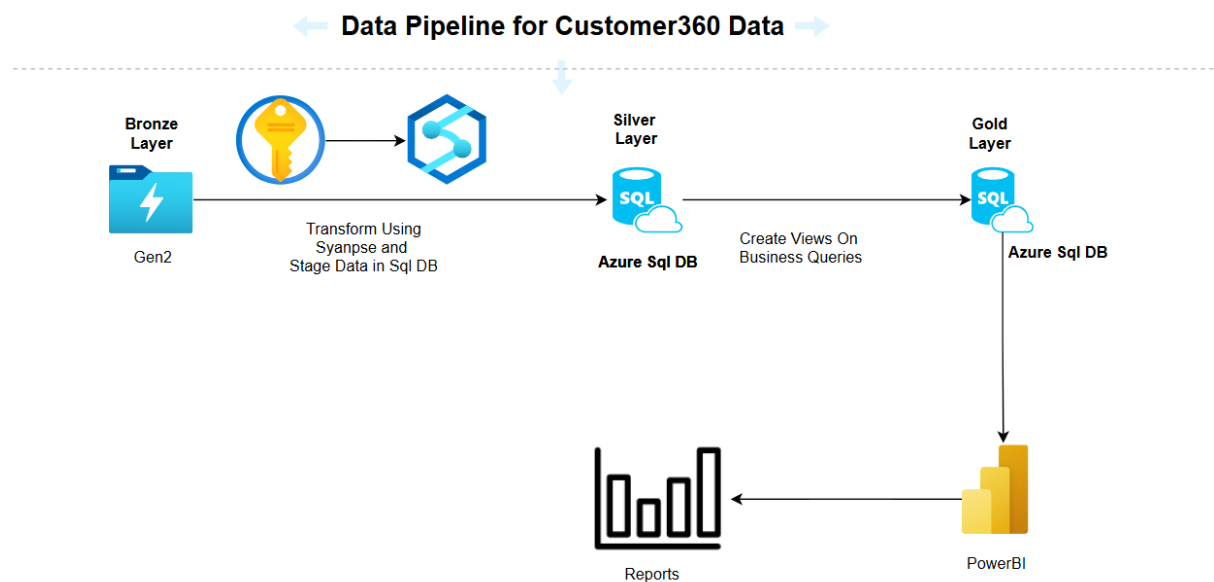
Table of Contents

1. [Project Overview](#)
2. [Project Architecture](#)
3. [Roles and Responsibilities](#)
4. [Tools and Technologies](#)
5. [Step-by-Step Process](#)
 - [Step 1: Ingest Data](#)
 - [Step 2: Define Staging \(Curated\) Schema](#)
 - [Step 3: Define Analytics \(Gold\) Schema](#)
 - [Step 4: Load Curated Data](#)
 - [Step 5: Build a Power BI Dashboard](#)
6. [Validation and Verification](#)
7. [Deliverables](#)

Project Overview

A retail business seeks to develop a unified Customer 360 view by integrating data from various sources: online transactions, in-store purchases, customer service records, and loyalty programs. The architecture employs fact and dimension tables for a scalable and organized structure.

Project Architecture



Tools and Technologies

- Azure Synapse Analytics
- Azure Data Lake Storage (ADLS)
- Azure SQL Database
- Power BI

Step-by-Step Process

Step 1: Ingest Data

- Utilize Azure Synapse pipelines to ingest data from multiple sources into the **Raw** container in ADLS.

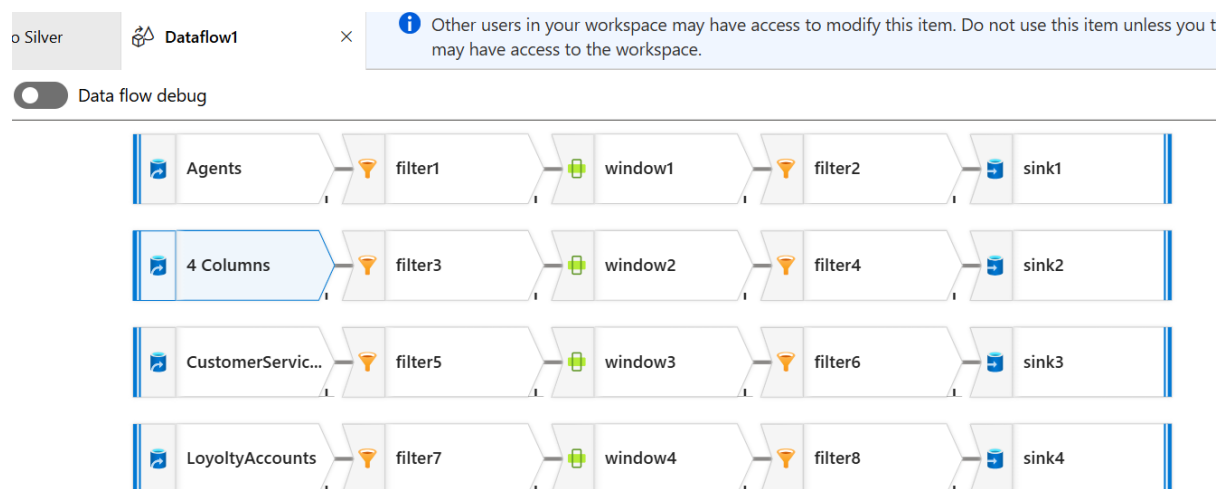
Location: input / Customer_360_Data

Search blobs by prefix (case-sensitive) ☐ Show deleted objects

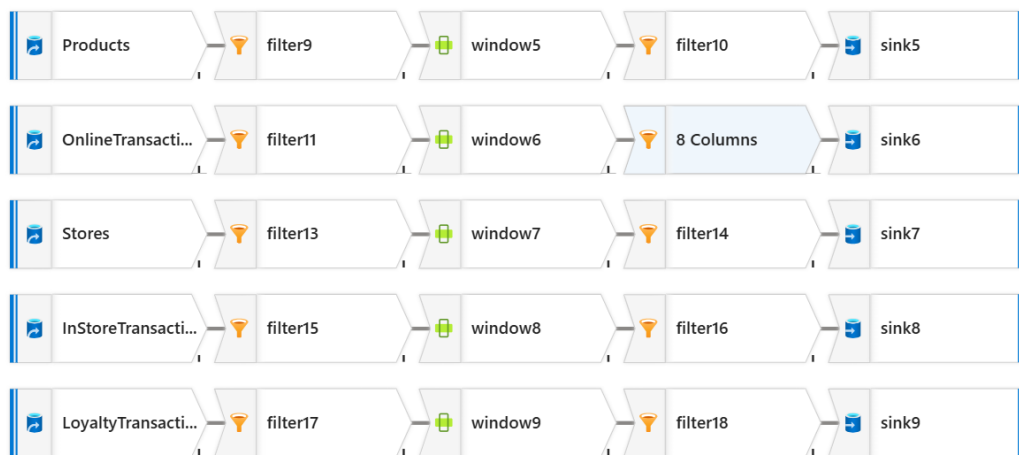
Name	Modified	Access tier	Archive status	Blob type	Size
☐ [.]					
☐ Agents.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	3.8
☐ Customers.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	8.7
☐ CustomerServiceInteractions.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	5 k
☐ InStoreTransactions.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	4.6
☐ LoyaltyAccounts.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	2.9
☐ LoyaltyTransactions.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	3.6
☐ OnlineTransactions.csv	4/26/2025, 5:37:28 PM	Hot (Inferred)		Block blob	5.4

Step 2: Define Staging (Curated) Schema

- Create Dataflows in Azure Synapse Analytics for the **Silver** layer to clean and standardize data and store it in Azure sql database.

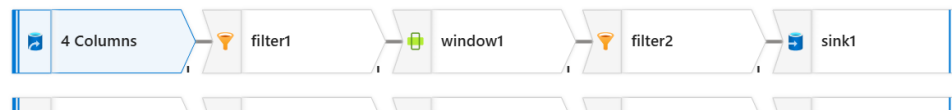


Data flow debug



Accounts dataflow

Source



Source settings **Source options** Projection Optimize Inspect Data preview

File settings

File mode ⓘ

☒ File ☐ Wildcard

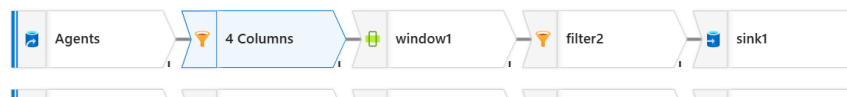
File path *

input / Customer_360_Data / Agents.csv [Browse](#)

Allow no files found ⓘ



Remove nulls using filter



Filter settings **Optimize** Inspect Data preview

Output stream name *

filter1

[Learn more](#)

Description

Filtering rows using expressions on columns 'AgentID'

[Reset](#)

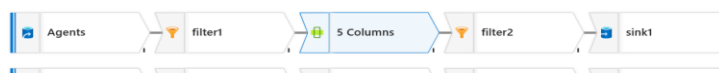
Incoming stream *

Agents

Filter on *

!isNull(AgentID)

Remove duplicate records based on major columns Using window transformation



Window settings Optimize Inspect Data preview

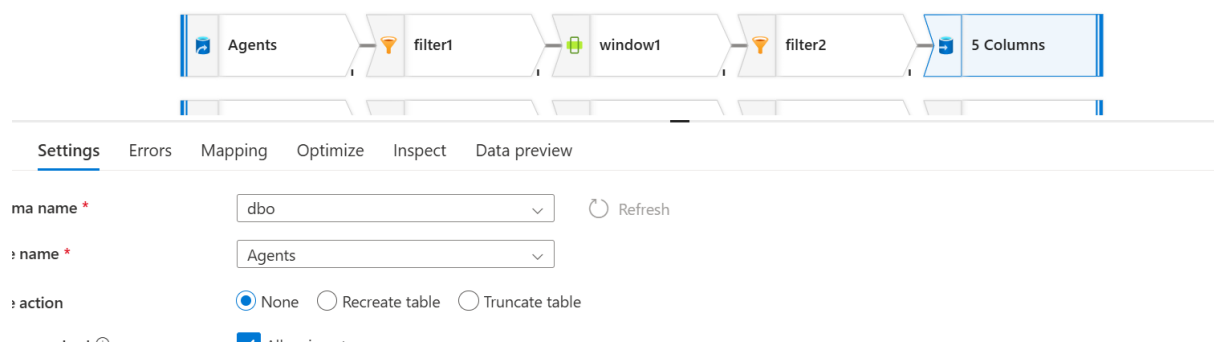
Incoming stream *

filter1

1. Over 2. Sort 3. Range by 4. Window columns

filter1's column	Name as	
123 AgentID	AgentID	+ -
abc Name	Name	+ -
abc Department	Department	+ -

Sink is sql database table



Similarly transformed all the other files in silver layer

Step 3: Define Analytics (Gold) Schema

My Sink Tables

CREATE TABLE Customers (

CustomerID INT PRIMARY KEY,

Name VARCHAR(100),

Email VARCHAR(100),

Address VARCHAR(255)

);

CREATE TABLE Products (

ProductID INT PRIMARY KEY,

Name VARCHAR(100),

Category VARCHAR(50),

Price DECIMAL(10, 2)

);

CREATE TABLE OnlineTransactions (

OrderID INT PRIMARY KEY,

CustomerID INT,

ProductID INT,

DateTime DATETIME,

```
PaymentMethod VARCHAR(50),  
Amount DECIMAL(10, 2),  
Status VARCHAR(20),  
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

```
CREATE TABLE Stores (  
    StoreID INT PRIMARY KEY,  
    Location VARCHAR(100),  
    Manager VARCHAR(100),  
    OpenHours VARCHAR(50)  
);
```

```
CREATE TABLE InStoreTransactions (  
    TransactionID INT PRIMARY KEY,  
    CustomerID INT,  
    StoreID INT,  
    DateTime DATETIME,  
    Amount DECIMAL(10, 2),  
    PaymentMethod VARCHAR(50),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (StoreID) REFERENCES Stores(StoreID)  
);
```

```
CREATE TABLE Agents (  
    AgentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Department VARCHAR(50),  
    Shift VARCHAR(50)  
);
```

```
CREATE TABLE CustomerServiceInteractions (  
    InteractionID INT PRIMARY KEY,  
    CustomerID INT,  
    DateTime DATETIME,  
    AgentID INT,  
    IssueType VARCHAR(50),  
    ResolutionStatus VARCHAR(50),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (AgentID) REFERENCES Agents(AgentID)  
);
```

```
CREATE TABLE LoyaltyAccounts (  
    LoyaltyID INT PRIMARY KEY,  
    CustomerID INT,  
    PointsEarned INT,  
    TierLevel VARCHAR(20),  
    JoinDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE LoyaltyTransactions (  
    LoyaltyID INT,  
    DateTime DATETIME,  
    PointsChange INT,  
    Reason VARCHAR(100),  
    PRIMARY KEY (LoyaltyID, DateTime),  
    FOREIGN KEY (LoyaltyID) REFERENCES LoyaltyAccounts(LoyaltyID)  
);
```

Step 4: Created Views to Generate Business insights

View 1 - for Average Order Value (AOV)

SUM(Amount) / COUNT(OrderID) per product, category, and location.

```
create or Alter View Average_Order_Value as
select
    (sum(o.Amount)/COUNT(o.OrderID)) as Average_Order_Value,
    o.ProductID,
    p.Category,
    c.Address AS Location
from OnlineTransactions o
join Customers c on o.CustomerID=c.CustomerID
join Products p on p.ProductID=o.ProductID
Group By o.ProductID,p.Category,c.Address;
```

Output:


```
select * from Average_Order_Value
```

110 %

Results

Messages

	Average_Order_Value	ProductID	Category	Location
1	103.300000	73	Clothing	00659 Christopher Square, South Maryside, NH 16004
2	122.990000	54	Electronics	019 Jonathan Center Apt. 043, Simpsontown, HI 900...
3	90.370000	78	Clothing	02411 Ruiz Corners Apt. 346, Oconnorberg, SD 93666
4	195.760000	46	Books	0574 Johnson Ford Apt. 404, Hayesside, NC 59305
5	87.740000	53	Electronics	0574 Johnson Ford Apt. 404, Hayesside, NC 59305
6	143.570000	6	Books	06255 Ellis Inlet, Morrisfort, RI 47036
7	17.410000	41	Books	071 Thomas Branch Suite 866, Alexanderberg, SC 8...
8	177.550000	85	Toys	0842 Samantha Lights, Lake Latoyacheater, NV 99023
9	175.990000	99	Clothing	0842 Samantha Lights, Lake Latovacheater, NV 99023

View 2 (Customer Segments)

- for Segment customers based on total spend, purchase frequency, and loyalty tier (LoyaltyAccounts.TierLevel).

Example: "High-Value Customers" (Top 10% spenders), "One-Time Buyers," "Loyalty Champions."

Create or Alter View Customer_segments as

SELECT

c.CustomerID,

c.Name,

COALESCE(ca.Total_Spend,0) as Total_Spend,

COALESCE(ca.PurchaseFrequency,0) as PurchaseFrequency,

CASE

WHEN ca.HighestTierValue = 4 THEN 'Platinum'

WHEN ca.HighestTierValue = 3 THEN 'Gold'

WHEN ca.HighestTierValue = 2 THEN 'Silver'

WHEN ca.HighestTierValue = 1 THEN 'Bronze'

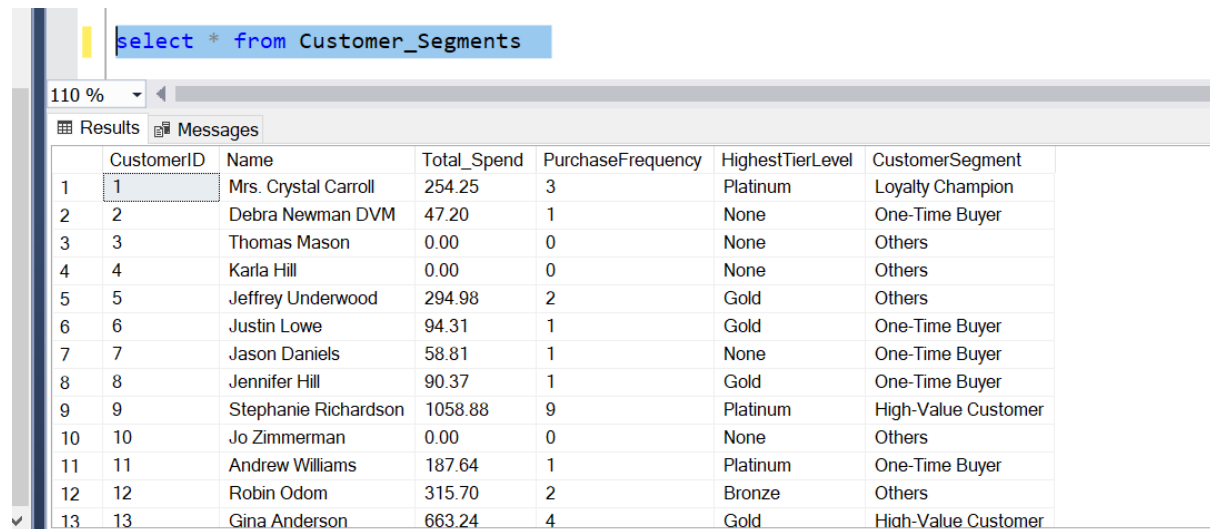
```

        ELSE 'None'
    END AS HighestTierLevel,
    CASE
        WHEN COALESCE(ca.Total_Spend, 0) >= 500 THEN 'High-Value
Customer'
        WHEN COALESCE(ca.PurchaseFrequency, 0) = 1 THEN 'One-Time
Buyer'
        WHEN ca.HighestTierValue IN (3, 4) AND
COALESCE(ca.PurchaseFrequency, 0) >= 3 THEN 'Loyalty Champion'
        WHEN COALESCE(ca.PurchaseFrequency, 0) > 2 THEN 'Frequent
Buyer'
        ELSE 'Others'
    END AS CustomerSegment
FROM
    Customers c
LEFT JOIN
    (SELECT
        o.CustomerID,
        SUM(o.Amount)AS Total_Spend,
        COUNT(o.OrderID) AS PurchaseFrequency,

        MAX(CASE
            WHEN l.TierLevel = 'Platinum' THEN 4
            WHEN l.TierLevel = 'Gold' THEN 3
            WHEN l.TierLevel = 'Silver' THEN 2
            WHEN l.TierLevel = 'Bronze' THEN 1
            ELSE 0
        END) AS HighestTierValue
    FROM
        OnlineTransactions o
    LEFT JOIN
        LoyaltyAccounts l ON o.CustomerID = l.CustomerID
    GROUP BY
        o.CustomerID) ca on ca.CustomerID=c.CustomerID

```

Output:



The screenshot shows a SQL query execution window. The query bar at the top contains the text: `select * from Customer_Segments`. Below the query bar, there are tabs for "Results" and "Messages". The "Results" tab is active, displaying a table with 7 columns: CustomerID, Name, Total_Spend, PurchaseFrequency, HighestTierLevel, and CustomerSegment. The table contains 13 rows of data, numbered 1 through 13 in the first column. The data is as follows:

	CustomerID	Name	Total_Spend	PurchaseFrequency	HighestTierLevel	CustomerSegment
1	1	Mrs. Crystal Carroll	254.25	3	Platinum	Loyalty Champion
2	2	Debra Newman DVM	47.20	1	None	One-Time Buyer
3	3	Thomas Mason	0.00	0	None	Others
4	4	Karla Hill	0.00	0	None	Others
5	5	Jeffrey Underwood	294.98	2	Gold	Others
6	6	Justin Lowe	94.31	1	Gold	One-Time Buyer
7	7	Jason Daniels	58.81	1	None	One-Time Buyer
8	8	Jennifer Hill	90.37	1	Gold	One-Time Buyer
9	9	Stephanie Richardson	1058.88	9	Platinum	High-Value Customer
10	10	Jo Zimmerman	0.00	0	None	Others
11	11	Andrew Williams	187.64	1	Platinum	One-Time Buyer
12	12	Robin Odom	315.70	2	Bronze	Others
13	13	Gina Anderson	663.24	4	Gold	High-Value Customer

View 3

- - To Analyze DateTime to find peak days and times in-store vs. online.

```
CREATE OR ALTER VIEW Peak_Days_Hours AS
WITH Onlinets AS (
    SELECT
        DATEPART(WEEKDAY, ots.DateTime) AS DayOfWeek,
        DATENAME(WEEKDAY, ots.DateTime) AS DayName,
        DATEPART(HOUR, ots.DateTime) AS HourOfWeek,
        COUNT(ots.OrderID) AS OnlineTransactions,
        SUM(ots.Amount) AS OnlineTotalSales
    FROM
        OnlineTransactions ots
    GROUP BY
        DATEPART(WEEKDAY, ots.DateTime),
        DATENAME(WEEKDAY, ots.DateTime),
        DATEPART(HOUR, ots.DateTime)
),
Instorets AS (
    SELECT
        DATEPART(WEEKDAY, ist.DateTime) AS DayOfWeek,
```

```

        DATENAME(WEEKDAY, ist.DateTime) AS DayName,
        DATEPART(HOUR, ist.DateTime) AS HourOfWeek,
        COUNT(ist.TransactionID) AS InStoreTransactions,
        SUM(ist.Amount) AS InStoreTotalSales
    FROM
        InStoreTransactions ist
    GROUP BY
        DATEPART(WEEKDAY, ist.DateTime),
        DATENAME(WEEKDAY, ist.DateTime),
        DATEPART(HOUR, ist.DateTime)
)

SELECT
    COALESCE(o.HourOfWeek, i.HourOfWeek) AS HourOfWeek,
    COALESCE(o.DayName, i.DayName) AS DayName,
    ISNULL(o.OnlineTransactions, 0) AS OnlineTransactions,
    ISNULL(o.OnlineTotalSales, 0) AS OnlineTotalSales,
    ISNULL(i.InStoreTransactions, 0) AS InStoreTransactions,
    ISNULL(i.InStoreTotalSales, 0) AS InStoreTotalSales
FROM
    Onlinets o
FULL OUTER JOIN Instorets i
    ON o.HourOfWeek = i.HourOfWeek AND o.DayOfWeek = i.DayOfWeek;

```

Output:

Instorets AS (						
110 %						
Results Messages						
	HourOfWeek	DayName	OnlineTransactions	OnlineTotalSales	InStoreTransactions	InStoreTotalSales
10	1	Saturday	1	123.91	0	0.00
11	2	Sunday	0	0.00	3	337.76
12	2	Monday	2	92.83	0	0.00
13	2	Tuesday	0	0.00	1	23.93
14	2	Wednesday	1	16.63	0	0.00
15	2	Saturday	1	27.55	1	69.63
16	3	Sunday	1	93.68	2	238.33
17	3	Tuesday	1	103.67	1	160.38
18	3	Saturday	2	105.15	2	163.91
19	4	Tuesday	1	192.49	2	110.62
20	4	Wednesday	2	215.71	1	180.01
21	4	Thursday	2	283.78	1	121.51

```

select DayName,sum(OnlineTransactions) as OnlineTransactions ,sum(OnlineTotalSales) as OnlineTotalSales
,sum(InStoreTransactions) as InStoreTransactions,sum(InStoreTotalSales) as InStoreTotalSales from Peak_Days_Hours
group by DayName

select * from Peak_Days_Hours

```

110 %

Results

Messages

	DayName	OnlineTransactions	OnlineTotalSales	InStoreTransactions	InStoreTotalSales
1	Friday	15	1521.96	12	1278.60
2	Monday	14	1900.91	14	1581.47
3	Saturday	10	835.34	13	1123.88
4	Sunday	14	1404.80	12	1245.88
5	Thursday	9	1012.27	14	1689.89
6	Tuesday	16	1924.86	19	1717.92
7	Wednesday	22	2756.18	16	1850.06

View 4 - Agent Performance

- for Number of interactions and resolution success rates per agent (ResolutionStatus).

```
CREATE OR ALTER VIEW AGENTS_SUCCESS AS
```

```
SELECT
```

```
    A.Name,
```

```
    COUNT(CS.InteractionID) AS Interactions_count,
```

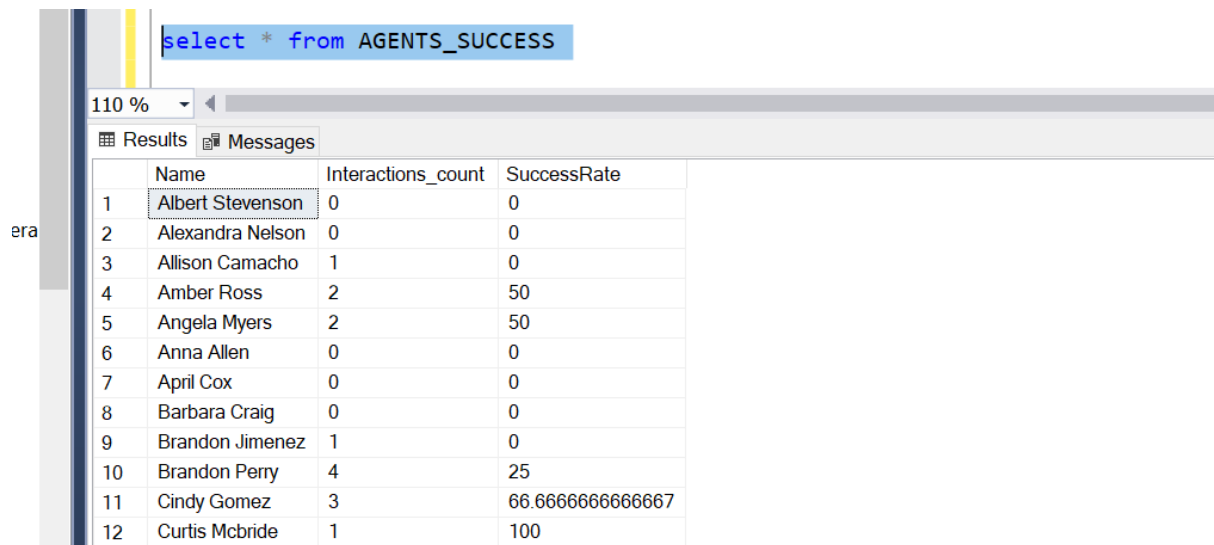
```
    COALESCE((CAST(COUNT(CASE WHEN CS.ResolutionStatus = 'Resolved' THEN 1
END) AS DECIMAL) /
```

```

        NULLIF(COUNT(CS.InteractionID), 0))*100,0) AS SuccessRate
FROM
    Agents A
LEFT JOIN
    CustomerServiceInteractions CS ON CS.AgentID = A.AgentID
GROUP BY
    A.Name;

```

Output:

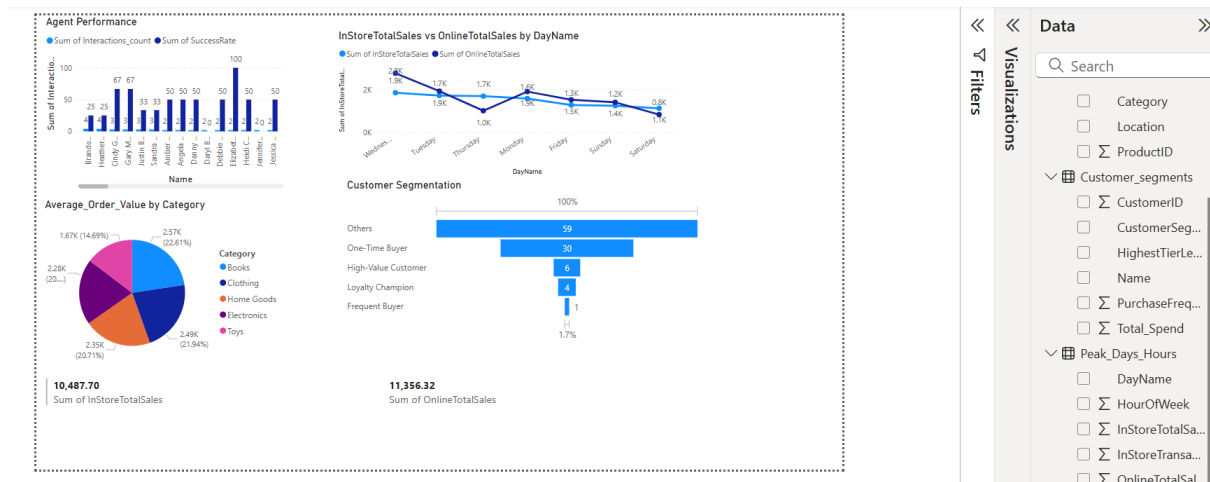


The screenshot shows a SQL query editor with the query `select * from AGENTS_SUCCESS` entered. Below the query, the 'Results' tab is active, displaying a table with 12 rows of data. The table has four columns: 'Name', 'Interactions_count', and 'SuccessRate'. The first row, 'Albert Stevenson', is highlighted. The 'SuccessRate' column contains values ranging from 0 to 100, with some decimal values like 66.6666666666667.

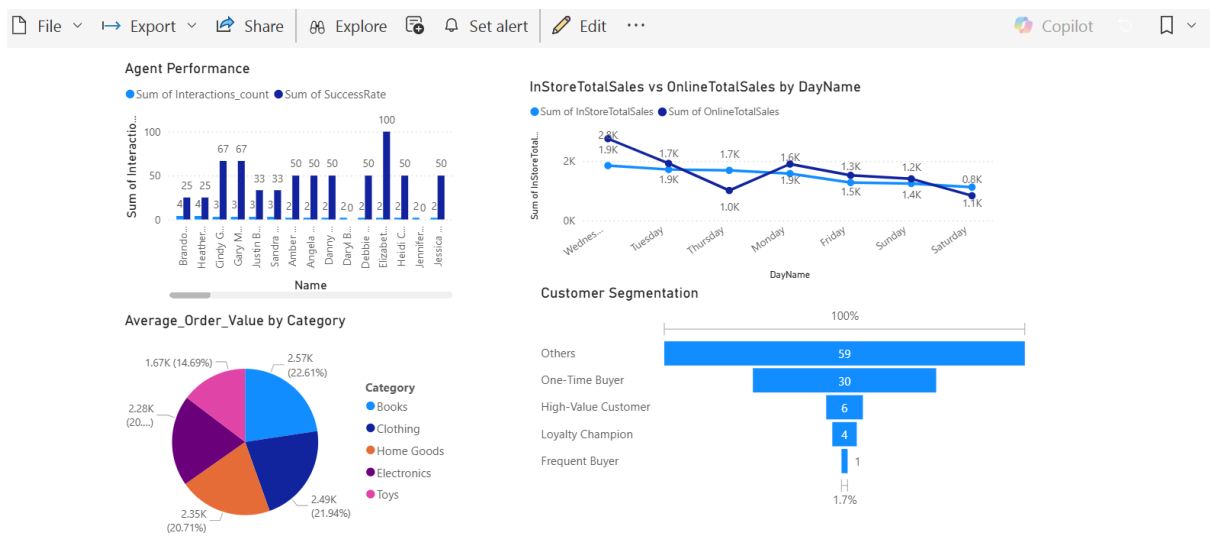
	Name	Interactions_count	SuccessRate
1	Albert Stevenson	0	0
2	Alexandra Nelson	0	0
3	Allison Camacho	1	0
4	Amber Ross	2	50
5	Angela Myers	2	50
6	Anna Allen	0	0
7	April Cox	0	0
8	Barbara Craig	0	0
9	Brandon Jimenez	1	0
10	Brandon Perry	4	25
11	Cindy Gomez	3	66.6666666666667
12	Curtis McBride	1	100

Step 5: Build a Power BI Dashboard

- Develop an interactive dashboard visualizing Customer 360 insights.



- Publish the dashboard to **Fabric Workspace**.



Deliverables

- **Ingested Data:** Raw datasets in ADLS.

- **Curated Data:** Cleaned and standardized data in Synapse (Silver).
- **Analytics Data:** Business-ready tables in Azure SQL Database (Gold).
- **Power BI Dashboard:** Final report with interactive visuals.

Documentation and Upload Guidelines

Upload the following materials to GitHub:

- **SQL Scripts:** Both DDL and sample DML scripts.
- **Pipeline JSON:** Exported from Azure Synapse pipelines.
- **Documentation:** A README.md or .doc file explaining the process, architecture, and logic.

Github: https://github.com/VishalKanaka/Customer360-Tranformation-DataAnalysis_Using-AzureSynapse-Azure-sql-and-PowerBI.git

Reference Dataset

Use realistic sample data from **Kaggle's Customer 360 Data** for testing and validation.